

Investigación sobre Panoptic Scene Graph Generation (PSG) como herramienta para image captioning y comparación con VLMs

Victoria Lopéz

Aprendizaje Automático para Datos en Grafos 2025

Facultad de Ingeniería

Montevideo, Uruguay

victorialopez4929@gmail.com

Uriel Yaffé

Aprendizaje Automático para Datos en Grafos 2025

Facultad de Ingeniería

Montevideo, Uruguay

urielyaffe@gmail.com

I. MOTIVACION

La extracción de información valiosa de una imagen ha sido un problema de gran interés tecnológico por décadas, pero la capacidad de comprender semánticamente una escena de la forma en que los humanos lo hacemos ha sido considerada el desafío definitivo de la visión por computadora. Si bien los problemas de detección de objetos han sido trabajados en profundidad, comprender una escena va más allá de la localización de objetos y se enfoca en una tarea de más alto nivel, poniendo énfasis en explorar el valioso significado semántico que implica la relación entre los objetos, así como la interacción de estos objetos con su entorno.

De esta forma, el reto fundamental reside en cómo pasar de una caracterización puramente visual (señalar objetos) a una comprensión cognitiva. Mientras que la detección de objetos identifica “qué” hay en la imagen, el entendimiento de la escena requiere descifrar “cómo” interactúan esos elementos. Esta brecha entre la percepción visual y la percepción semántica es lo que la generación de grafos de escena (*Scene Graph Generation* - SGG) busca cerrar, convirtiendo una imagen o un video en una representación estructurada compuesta por objetos, sus atributos y las relaciones que los unen.

En este informe buscamos presentar de forma incremental cómo obtener una descripción de la escena de una imagen mediante el uso de grafos. El recorrido parte desde los fundamentos del *Scene Graph Generation* (SGG) hasta centrarse en el modelo *Panoptic Scene Graph* (PSG), el cual exploraremos en detalle por su capacidad para integrar la segmentación panóptica con la generación de relaciones semánticas.

Dicho esto, el auge de los modelos multimodales masivos, como pueden ser los *VLMs*, ha planteado la interrogante sobre la necesidad de generar grafos explícitos. Si un modelo multimodal puede describir una escena directamente en texto, ¿tiene mejores resultados que un enfoque basado en grafos? Esta es una de las interrogantes que buscaremos responder, para luego dar una valoración de aquellos casos donde tener un grafo que describa una escena nos habilita la resolución de problemas que un *VLM* no permite.

Por último, comentaremos sobre el estado del arte en segmentación panóptica: *Open-set Panoptic Scene Graph Generation* [4] y qué aporta respecto a PSG.

II. FUNDAMENTOS: SCENE GRAPH GENERATION (SGG) TRADICIONAL

El objetivo de *Scene Graph Generation* (SGG) es representar una imagen mediante una estructura semántica en forma de grafo. En esta representación, los nodos corresponden a los objetos detectados en la escena, mientras que las aristas describen las relaciones entre ellos. La unidad básica de este modelo son los tripletes $\langle \text{sujeito}, \text{relación}, \text{objeto} \rangle$, los cuales permiten capturar las interacciones visuales presentes en la imagen.

Para modelar la distribución condicional de un grafo de escena G dado una imagen de entrada I , se define:

$$\Pr(G | I) = \Pr(B, O, R | I) \quad (1)$$

donde $I \in \mathbb{R}^{H \times W \times 3}$ representa la imagen de entrada como un tensor tridimensional de altura H , ancho W y tres canales de color.

El grafo de escena G está compuesto por tres conjuntos principales:

- **Bounding boxes:** $B = \{b_1, \dots, b_n\}$, donde cada $b_i \in \mathbb{R}^4$ representa las coordenadas espaciales de un objeto en la imagen.
- **Clases de objetos:** $O = \{o_1, \dots, o_n\}$, donde cada $o_i \in \mathcal{C}_O$ corresponde a la etiqueta semántica de un objeto, perteneciente a un conjunto finito de clases $\mathcal{C}_O$.
- **Relaciones:** $R = \{r_1, \dots, r_l\}$, donde cada $r_k \in \mathcal{C}_R$ representa una relación semántica entre pares de objetos, perteneciente al conjunto de relaciones $\mathcal{C}_R$.

Existen dos enfoques principales para abordar este problema:

a) **Two-stage:** En este enfoque, primero se detectan los objetos (mediante *Bounding boxes* y sus respectivas clases) y luego se predicen las relaciones entre pares de objetos. La probabilidad del grafo se descompone en etapas sucesivas

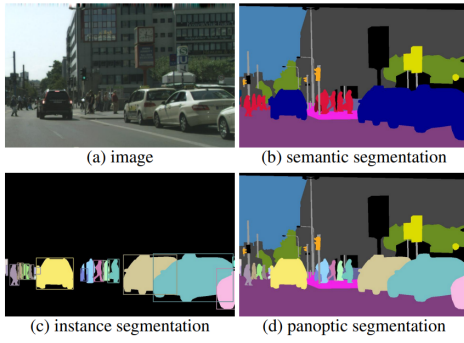


Fig. 1. Ejemplo de distintas segmentaciones. [3]

que incluyen detección de regiones, clasificación de objetos, predicción de atributos y modelado de relaciones.

b) One-stage: En este enfoque, la detección de objetos y la predicción de relaciones se realizan de manera conjunta mediante un único modelo, lo que simplifica el *pipeline* y permite una inferencia más eficiente.

III. SEGMENTACIÓN PANÓPTICA COMO SOLUCIÓN A LAS LIMITACIONES DE SGG

Como se mencionó en la sección anterior, el problema de definir un escenario, con sus objetos, predicados y personas, tiene una componente importante en definir cada objeto o persona para generar dichas relaciones. En particular, SGG utiliza *Bounding Boxes*.

En los métodos basados en *bounding boxes*, cada objeto es delimitado mediante un rectángulo que aproxima su ubicación dentro de la imagen. Si bien esta representación es eficiente y ampliamente utilizada, las cajas delimitadoras no capturan con precisión la forma real del objeto y suelen incluir píxeles que no pertenecen al mismo, especialmente en escenas complejas o con objetos parcialmente superpuestos.

La segmentación panóptica combina la segmentación semántica y la segmentación de instancias.

La segmentación semántica asigna una etiqueta de clase a cada píxel de la imagen, mientras que la segmentación de instancias distingue entre diferentes objetos pertenecientes a la misma categoría.

La segmentación panóptica permite identificar cada objeto individual presente en la imagen y marcar exactamente los píxeles que lo componen. En el contexto de PSG, cada objeto segmentado se representa mediante una máscara binaria que indica qué píxeles pertenecen al objeto, lo que resulta en una representación más detallada de la escena que la obtenida con las *bounding boxes*.

IV. EL SALTO A PANOPTIC SCENE GRAPH (PSG)

La *Panoptic Scene Graph Generation* (PSG) surge como una mejora al problema de *Scene Graph Generation* (SGG). Como primer cambio, se utiliza la segmentación panóptica en lugar de las *bounding boxes*.

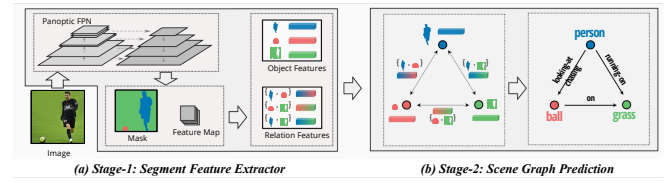


Fig. 2. Two-Stage Baseline [4]

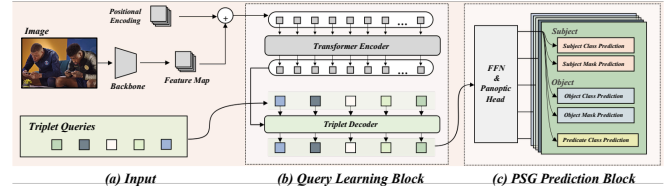


Fig. 3. Estructura PSGTR [4]

La mayoría de los métodos propuestos en este contexto se desarrollan bajo un esquema de conjunto cerrado (*closed-set*), donde **todas las categorías posibles están previamente definidas durante la etapa de entrenamiento**.

Para describir PSG, tomaremos como referencia el trabajo de [4], el cual presenta distintas arquitecturas para abordar el problema de *Panoptic Scene Graph Generation*. En particular, al igual que en SGG, el artículo propone dos enfoques principales: *One-Stage Baseline* y *Two-Stage Baseline*.

A. Two-Stage Baseline

a) *Two-Stage Baseline*: El *baseline* de la implementación de PSG de dos etapas se ilustra en la Figura 2.

En la primera etapa, se utiliza un segmentador panóptico preentrenado que, dado como entrada una imagen, produce máscaras panópticas, predicciones de clases para cada región y las correspondientes *features* visuales. Estas *features* suelen capturar información de apariencia, posición espacial y, en algunos casos, contexto local de cada objeto detectado.

En la segunda etapa, la información generada previamente se emplea como entrada de un predictor de relaciones. Este módulo, típicamente heredado de modelos de *Scene Graph Generation* (SGG), evalúa pares de objetos y predice la relación semántica entre ellos. Para ello, utiliza representaciones conjuntas de los objetos, como la unión de sus regiones o combinaciones de sus *features*, con el fin de modelar las interacciones entre los mismos.

B. One-staged Baseline

1) *PSGTR*: *Panoptic Scene Graph Transformer* propone un enfoque basado en la arquitectura *Transformer*, cuyo objetivo es predecir de manera conjunta tanto las tripletas de la escena como la localización de los objetos, sin requerir una etapa previa e independiente de segmentación.

Su estructura se observa en Figure 3.

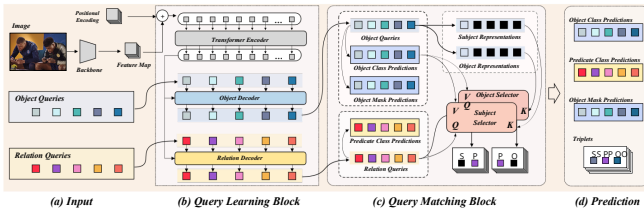


Fig. 4. Estructura PSGFormer [4]

A diferencia de los enfoques *two-stage*, PSGTR modela directamente la distribución conjunta del grafo de escena a partir de la imagen mediante un conjunto fijo de *queries* aprendidas. Cada *query* es responsable de predecir una entidad o una relación, permitiendo así una inferencia global y paralela.

Recordando de la section II, el modelo aprende una función que aproxima:

$$\Pr(G \mid I) = \Pr(M, O, R \mid I) \quad (2)$$

donde M corresponde a las máscaras panópticas, O a las clases de objetos y R a las relaciones entre ellos.

Esta predicción se realiza de manera conjunta a través de un *decoder* tipo *Transformer*, que recibe como entrada un conjunto de *queries* $\{q_i\}$ y *features* visuales extraídas por un *backbone* convolucional.

El mecanismo central del modelo se basa en la *self-attention*, que permite modelar las interacciones entre las distintas *queries*:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \quad (3)$$

donde Q , K y V corresponden a proyecciones lineales de las *queries* y las *features* de la imagen.

- Q (Query): representa qué información está buscando cada elemento.
- K (Key): representa qué información contiene cada elemento disponible para ser consultado.
- V (Value): representa el contenido que será agregado o combinado en la salida según la atención calculada.

Este mecanismo permite capturar dependencias globales entre objetos y relaciones, evitando la necesidad de evaluar explícitamente todos los pares posibles.

2) *PSGFormer*: PSGFormer introduce un modelado basado en *prompts*, separando la detección de objetos de la detección de relaciones.

Para ello, el modelo utiliza *queries* aprendidas dentro del *Transformer*. En particular, las *queries* de relaciones pueden interpretarse como *prompts*, ya que guían la búsqueda de interacciones relevantes en la imagen.

Estas *queries* interactúan con las *features* visuales mediante mecanismos de atención, permitiendo al modelo identificar la información más relevante para cada tarea.

La arquitectura puede dividirse en cuatro etapas principales: extracción de características de la imagen, *Query*

Learning Block, *Query Matching Block* y finalmente la etapa de predicción de las tripletas. Su estructura se presenta en Figure 4

a) *Extracción de características*: Dada una imagen de entrada I , primero se obtiene un mapa de características utilizando un *backbone* convolucional.

$$F = \text{Backbone}(I)$$

Donde $F \in \mathbb{R}^{H \times W \times C}$ representa el *feature map*. A estas representaciones se les agrega un *positional encoding* para preservar la información espacial.

Posteriormente, estas representaciones son procesadas por un *Transformer Encoder*, el cual permite capturar dependencias globales entre diferentes regiones de la imagen mediante el mecanismo de atención de la Equation 3

b) *Query Learning Block*: En esta etapa, el modelo introduce dos conjuntos de *queries* aprendidas:

- *Object queries*
- *Relation queries*

Cada conjunto de *queries* es procesado por un *decoder* específico dentro del *Transformer*. En el caso de las *object queries*, el *Object Decoder* utiliza atención cruzada con el *feature map* para generar representaciones que contienen la información necesaria para predecir tanto la clase del objeto como su máscara de segmentación.

Por otro lado, las *relation queries* son procesadas por el *Relation Decoder*, generando representaciones las cuales capturan la información necesaria para identificar el tipo de relación presente en la escena.

c) *Query Matching Block*: Dado que objetos y relaciones se predicen de forma paralela e independiente, el modelo debe construir las triplas (sujeto, relación, objeto). Este proceso se formula como un problema de *prompting*.

Cada *relation query* actúa como un *prompt* que busca, dentro del conjunto de objetos detectados, aquellos que participan en la interacción.

Para permitir esta asignación, cada representación de objeto se proyecta mediante redes *feed-forward* diferentes para cada rol.

La compatibilidad entre una *relation query* y un objeto se evalúa mediante similitud del coseno

$$\text{sim}(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$$

Se calcula la similitud con todos los posibles sujetos y objetos. Los elementos con mayor similitud se seleccionan como candidatos para formar la tripleta correspondiente.

d) *Predicción*: En la etapa final, las representaciones generadas por los *decoders* se transforman en predicciones semánticas mediante cabezas de clasificación y segmentación. En particular, cada *embedding* se proyecta mediante capas lineales para estimar probabilidades sobre las clases de objetos y de predicados, utilizando una función *softmax*

$$P(y|h) = \text{softmax}(Wh + b)$$

donde $h \in \mathbb{R}^d$ es el *embedding* producido por el *decoder*, $W \in \mathbb{R}^{C \times d}$ es una matriz de pesos aprendida que proyecta la representación al espacio de clases, $b \in \mathbb{R}^C$ es un vector de sesgo, y C es el número de clases posibles.

La función *softmax* convierte estos puntajes en una distribución de probabilidad sobre las clases.

Una vez obtenidas estas representaciones, el modelo construye las tripletas que describen la escena.

Para ello, cada *relation query* selecciona los objetos que participan en la interacción utilizando medidas de similitud entre *embeddings* permitiendo formar tripletas de la forma

(*sujeto*, *predicado*, *objeto*)

V. BREVE CONTEXTO SOBRE LOS VISION LANGUAGE MODELS

Antes de introducirnos en la parte puramente experimental, se realizó un breve repaso de cómo funcionan los *VLMs*. Los *Vision-Language Models* (VLMs) son sistemas de inteligencia artificial generativa y multimodal diseñados para comprender y procesar tanto imágenes como texto. A diferencia de los modelos tradicionales de *computer vision*, que están limitados a tareas específicas como la detección o clasificación de objetos, los *VLMs* pueden abordar diversas tareas a partir de instrucciones en lenguaje natural.

Esencialmente, su arquitectura se compone de tres bloques:

- **Codificador de visión**, que asocia imágenes con texto. Utiliza un modelo basado en *transformers* entrenado sobre millones de pares imagen-texto.
- **Proyector**, que traduce las señales visuales generadas por el codificador en *tokens* de imagen comprensibles para un *LLM*.
- **Large Language Model (LLM)**, que procesa toda la información para generar una respuesta en lenguaje natural.

Si bien la gran cantidad de datos con los que estos modelos fueron entrenados les otorga una versatilidad significativa, una de sus principales limitaciones es la **comprensión espacial y la localización precisa de objetos en una imagen**. Esto se debe a que los codificadores de visión más comunes (como *CLIP*) se entrenan mayoritariamente con descripciones de texto cortas que no incluyen información espacial detallada.

En relación con esto, también presentan limitaciones de resolución y pérdida de detalles, ya que los modelos suelen estar restringidos a tamaños de entrada relativamente pequeños, como 448×448 . Al recibir imágenes de alta definición, resulta necesario realizar un reescalado, lo que conlleva la pérdida de detalles finos, como objetos pequeños o relaciones densas.

Dadas estas limitaciones intrínsecas de resolución, se pueden identificar escenarios que resultan desafiantes para un *VLM*, pero en los cuales se esperaría que un modelo basado en *PSG* obtenga mejores resultados. En la siguiente sección se explora este aspecto.

VI. EXPERIMENTACIÓN: COMPARAMOS LA SALIDA DE PSG CON LA DESCRIPCIÓN HECHA POR UN VLM

Los autores del artículo [4] disponibilizaron un repositorio de Github completo y bien estructurado, que incluye código

para entrenar y hacer inferencia sobre el modelo de *PSG* propuesto en el artículo.

Adicionalmente, los autores incluyen el *checkpoint* al mejor entrenamiento (correspondiente a los resultados que reportan), junto con la referencia a las imágenes del *dataset* de *COCO* [1] usado por ellos. Con esto, nos propusimos intentar hacer inferencia sobre un conjunto de imágenes para validar el funcionamiento del modelo.

A. Configuración inicial y prueba del modelo

Si bien el repositorio estaba debidamente documentado y contaba con archivos para configurar el entorno necesario para su uso, el principal inconveniente fue la compatibilidad de versiones de los paquetes requeridos. En particular, las versiones compatibles de *Python*, *Torch* y *MMCV* (*Multimedia Computer Vision*, que constituye la base de la librería *OpenPSG* desarrollada por los autores) eran bastante antiguas. Esto representó un problema, especialmente al intentar utilizar el mismo entorno para realizar inferencia con el modelo de lenguaje, el cual no admitía versiones tan desactualizadas de *Python* y *Torch*.

Asimismo, fue necesario realizar algunos ajustes en el código para habilitar la inferencia utilizando CPU, así como llevar a cabo un proceso mínimo de *debugging*. No obstante, más allá de estas modificaciones, fue posible utilizar los pesos provistos por los autores para realizar inferencia.

De esta manera, al tomar una imagen como entrada, esta era segmentada, se generaban y almacenaban sus máscaras panópticas, y las tripletas resultantes eran mostradas en la terminal.

Una vez que se logró tener el modelo base (*PSG*) funcionando, se quiso comparar su salida con la que puede llegar a darnos *VLM*.

B. Pipeline de comparación

Para poder evaluar las descripciones generadas por *PSG* y *VLM* fue necesario compararlas con un *ground truth*, es decir, con descripciones que tomaremos como verídicas. Con esto, buscaremos evaluar qué tanto se acerca cada modelo a la descripción ideal. En nuestro caso, estas etiquetas son las asociadas al *dataset* de *PSG* generado por los autores del artículo descrito en capítulos anteriores, el cual se basa en el *dataset* de *COCO*.

Ahora bien, es importante recordar que las salidas de ambos modelos viven en espacios muy diferentes. Por un lado, *PSG* tiene como salida un grafo con objetos y sujetos como nodos y relaciones como aristas, mientras que un *VLM* tiene como salida un texto plano, por lo que antes de comparar los dos modelos será necesaria una etapa de normalización de las salidas.

Para ello se propuso el siguiente *pipeline* de comparación 5.

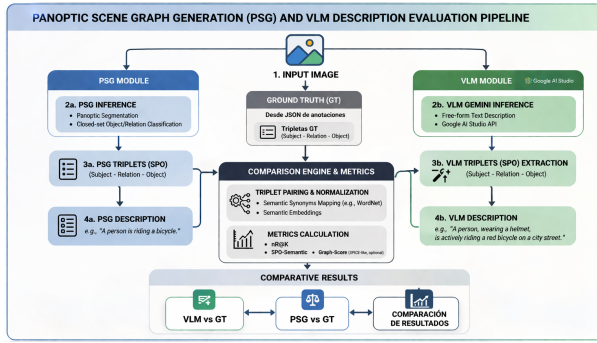


Fig. 5. Pipeline de evaluación de modelos.

El proceso parte de recibir una imagen como entrada y obtener la tripleta GT asociada a ella. Luego de esto, el *pipeline* se bifurca en una rama asociada al modelo *PSG* y otra asociada al *VLM*.

1) *Rama A: PSG*: La primera etapa de esta rama es hacer inferencia sobre la imagen con el modelo *PSG* usando el archivo de pesos brindado por los autores. El resultado de esta inferencia es, por un lado, las máscaras panópticas y por el otro, un objeto específico de la librería *openpsg* análogo a un grafo, con atributos correspondientes a los *IDs* de los objetos detectados (los nodos del grafo), las relaciones (las etiquetas de las aristas) y los pares objeto-objeto (las aristas en sí mismas).

El siguiente bloque (referenciado como el 3.a en la figura 5) se encarga de generar las tripletas (*sujeto*, *predicado*, *objeto*). Para ello, se obtuvieron las *top-k* relaciones sujeto-objeto con mayor *score* ($k = 10$ por *default*) y, a partir de estas, se construyó el objeto que posteriormente se utilizó para la comparación, el cual tiene la forma siguiente

```
1 [language=Python, caption=Ejemplo de inferencia con
  OpenPSG]
2 "triplets": [
3   {
4     "subject_label": "rug-merged",
5     "relation_label": "on",
6     "object_label": "floor-other-merged",
7     "score": 0.9758593440055847,
8     "subject_index": 0,
9     "object_index": 17
10  },
11  {...},
12  ...
13 ]
```

En este objeto se incluyen los componentes de la tripleta, los índices de cada uno y el *score* asociado a la tripleta en sí misma. En este punto, se decidió dar un paso adicional e intentar obtener una oración descriptiva a partir de las tripletas. La idea fue generar una oración simple que facilitara el entendimiento de la escena para un lector humano y que, a su vez, resultara más sencilla de comparar con la descripción generada por el *VLM*.

Para construir estas oraciones, se definieron algunas reglas gramaticales simples adaptadas al español, aunque cabe destacar que el procesamiento se hizo originalmente en inglés, ya que los modelos de *embeddings* presentan un mejor de-

sempño en dicho idioma. Si el predicado terminaba en *ing* (indicando una acción en progreso) o correspondía a una preposición, la oración adoptaba la forma “*El SUJETO está RELACIÓN el OBJETO*”. Por otro lado, si la relación contenía los verbos *has* o *have*, la estructura tomaba la forma “*El SUJETO RELACIÓN el OBJETO*” (por ejemplo, “El sujeto tiene el objeto”).

En este punto, consideramos importante no utilizar un *LLM* para generar las frases, ya que esto podría introducir capacidades propias del *VLM* en la inferencia, “contaminando” la comparación.

2) *Rama B: VLM*: Por otro lado, se utilizó la imagen de entrada como *input* para un *VLM* con el objetivo de obtener una descripción en texto libre de la misma. Para esta tarea, optamos por usar el modelo *gemini-2.5-flash* [2], ya que presentaba buenos resultados en visión, alta velocidad y disponibilidad como solución *open-source* a través de la API de *Google AI Studio*.

Para generar las descripciones, empleamos el siguiente *prompt*:

Estructura del Prompt de Inferencia (VLM)

“Describe the image briefly in one paragraph, then list all relationships in the format: subject - relation - object. Return the paragraph first, then a blank line, then the list.”

De esta forma, se englobaron los bloques 3.b y 4.b detallados en la Figura 5 en uno solo, obteniendo tanto la descripción de la imagen en lenguaje natural como la lista de tripletas (*sujeto*, *predicado*, *objeto*).

En este punto, se contó con un conjunto de tripletas (*sujeto*, *predicado*, *objeto*), junto con descripciones en texto libre generadas por ambos modelos, así como los *ground truth* correspondientes a un conjunto de imágenes del *dataset* de *PSG*. El objetivo fue definir una o más métricas que permitieran evaluar cuantitativamente la calidad de estas descripciones.

Antes de proceder al cálculo de las métricas, resultó importante considerar que los *VLMs* presentan una naturaleza inherentemente probabilística, lo que implica que las descripciones generadas no son determinísticas. En particular, es esperable el uso de sinónimos, por lo que se consideró necesario incorporar una etapa de normalización previa al cálculo de las métricas, con el fin de reducir la variabilidad en las salidas del *VLM* y mejorar la precisión de la comparación.

Para ello, se optó por una normalización de términos que consistió en convertir el texto a minúsculas y eliminar sufijos presentes en la base (como “-merged” o “-other”). Asimismo, se implementó un mapeo de sinónimos en el que términos como “man”, “woman”, “boy” y “girl” fueron agrupados bajo el término “person”.

Es importante destacar que este mapeo se realizó de forma algorítmica utilizando listas predefinidas, sin recurrir a un *LLM*. Al igual que en la subsección anterior, esta decisión tuvo como objetivo evitar cualquier tipo de “contaminación”

o sesgo en las tripletas que posteriormente serían comparadas. La principal desventaja de este enfoque radica en la imposibilidad de cubrir la totalidad de sinónimos presentes en descripciones abiertas; por lo tanto, aquellas palabras que no figuren en el mapeo no serán preprocesadas, lo que podría penalizar la métrica de similitud al no reconocer equivalencias semánticas fuera del diccionario.

Luego de este preprocesamiento, se procedió a evaluar las inferencias realizadas por ambos modelos. En la siguiente sección se detallan las métricas utilizadas para este propósito.

3) *Métricas calculadas*: La primera métrica que se consideró fue la presentada en el artículo [4]: *mR@K*, la cual mide el *recall* medio de las *top-k* tripletas predichas por cada modelo. En este contexto, el *recall* cuantifica la proporción de tripletas relevantes del *ground truth* que son correctamente recuperadas por el modelo. Nos pareció adecuado adoptar la misma métrica utilizada por los autores, no solo por su pertinencia para este tipo de tarea, sino también con el objetivo de verificar en qué medida es posible reproducir los resultados reportados sobre el conjunto de test.

Buscando complementar este análisis, se planteó una prueba exploratoria sencilla, aplicando ambos modelos sobre una misma imagen y comparando directamente las descripciones generadas. La imagen analizada fue la de la figura 6.



Fig. 6. Imagen a describir por ambos modelos

A partir de esta imagen se obtuvieron las siguientes descripciones.

Respuesta generada por el VLM

"This outdoor image captures a unique wedding moment on a street, where a woman in a grey top holds a white umbrella over a groom in a tuxedo. The groom is receiving a white and green bouquet from his bride, who is dressed in a white wedding gown and veil, as they both stand amidst what appears to be a public event setting with portable toilets and tents in the background."

Respuesta generada por el PSG

"The person is wearing the tie. The person standing on the road. The person walking on the road. The person standing on the road. The person is holding the umbrella. The person walking on the road. The sky-other-merged is over the tree-merged. The person is wearing the tie. The person is holding the umbrella. The person is carrying the handbag. The person is holding the umbrella. The sky-other-merged is over the building-other-merged. The person walking on the road. The person is holding the umbrella. The sky-other-merged is over the road. The person is beside the person. The sky-other-merged is over the person. The person standing on the road. The sky-other-merged is over the person. The road is attached to the pavement-merged. The person is carrying the handbag. The umbrella is over the person. The person is carrying the handbag. The person is carrying the handbag. The tree-merged is beside the building-other-merged. The person is beside the person. The person walking on the road. The person is beside the person. The sky-other-merged is over the person. The person is holding the umbrella. The sky-other-merged is over the umbrella. The person standing on the pavement-merged. The person is beside the person. The pavement-merged is attached to the road. The pavement-merged is attached to the road. The person looking at the person. The umbrella is over the person. The person is in front of the person. The person is beside the person. The sky-other-merged is over the person. The person standing on the road. The person standing on the road. The tree-merged is beside the road."

La primera diferencia que se observó entre ambas descripciones fue que la generada por *PSG* resultó significativamente más extensa, aunque no debido a un mayor nivel de detalle en los objetos o sus relaciones, sino a la mayor cantidad de relaciones incluidas. En contraste, la descripción generada por el *VLM* fue más verbosa en cuanto a los atributos de los objetos identificados, lo cual aportó poco en términos de la comprensión espacial de la imagen.

En segundo lugar, se observó que, si bien existía una relación semántica entre ambas predicciones, las palabras utilizadas no coincidían. Tomando como ejemplo la descripción del *VLM*: "woman in a grey top holds a white umbrella", y comparándola con la descripción generada por *PSG* para la misma relación: "The person is holding the umbrella", se evidenció esta diferencia.

En este caso, si se evaluarán las tripletas asociadas a las

frases anteriores, presentadas en la Tabla II, utilizando la métrica $mR@1$, se obtendría un valor de 1 para *PSG* y 0 para el *VLM*, a pesar de que ambas descripciones transmiten una idea similar. Aunque las palabras no sean exactamente iguales, ambas capturan de manera consistente la interacción presente en la imagen. Por este motivo, resultó deseable definir una métrica que permitiera reflejar esta similitud semántica.

Fuente	Tripleta (S, P, O)
PSG	('person', 'holding', 'umbrella')
VLM	('left', 'holds', 'umbrella')
Ground Truth	('person', 'holding', 'umbrella')

TABLE I

COMPARACIÓN DE TRIPLETAS *ground truth* Y GENERADAS POR *PSG* Y *VLM*

Con esto en mente, se propuso la métrica ***SPO Semantic***, la cual calcula la similitud semántica entre las descripciones, haciéndola robusta frente al uso de sinónimos por parte de los modelos.

Para su cálculo, se obtuvo un *embedding* de la descripción generada por el modelo y otro correspondiente a las etiquetas (generado como se describió en la Sección subsección VI-B1) utilizando el modelo *all-MiniLM-L6-v2*. Posteriormente, ambos vectores se compararon mediante la similitud coseno.

De esta forma, al aplicar la métrica *SPO-Semantic*, se obtuvieron los siguientes resultados para las frases mencionadas anteriormente.

Modelo	SPO-Semantic
PSG	1
VLM	0.68

TABLE II

METRICA SPO-SEMANTIC PARA PORCIÓN DE DESCRIPCIÓN HECHA POR *PSG* Y *VLM*

Como se observó, esta métrica ubicó a ambas frases del ejemplo anterior más cerca del *ground truth* en comparación con la métrica previa, representando de mejor manera la similitud que se buscaba capturar.

No obstante, las métricas anteriores evaluaron, por un lado, la igualdad exacta de las tripletas y, por otro, la similitud semántica de las descripciones. En este contexto, resultó de interés definir una métrica que combinara ambos enfoques, aprovechando la robustez de la similitud semántica y la simplicidad estructural de las tripletas.

Con esta motivación, se propuso la métrica ***Graph Score***, la cual se calculó a partir de las *top-k* tripletas con mayor puntuación. El método consistió en generar un *embedding* para cada componente de la tripleta de manera independiente. En esta representación, el sujeto y el objeto se modelaron como nodos, mientras que la relación se representó como una arista, permitiendo proyectar cada elemento del grafo en un espacio vectorial.

Posteriormente, para cada componente de las tripletas extraídas, se buscó su correspondencia semántica óptima con los *embeddings* del *ground truth* mediante la similitud del coseno. De esta forma, se identificaron los pares más cercanos

de *nodo_estimado*–*nodo_gt* y *arista_estimada*–*arista_gt*. Finalmente, el *score* global del grafo se definió como una combinación ponderada de las similitudes de cada componente, calculada mediante la siguiente expresión:

$$GS = (score_{sujeto} \cdot 0.4) + (score_{objeto} \cdot 0.4) + (score_{relacion} \cdot 0.2) \quad (4)$$

Una vez planteadas estas tres métricas, procedimos a evaluar el desempeño de los modelos. En la siguiente sección, se presentan los resultados obtenidos.

C. Evaluación y comparación del rendimiento de los modelos

Para la fase de evaluación, se seleccionó un subconjunto de 20 imágenes del conjunto de prueba proporcionado por los autores de [4], el cual utiliza como base el *dataset* abierto *COCO*.

Uno de los principales desafíos durante este proceso, y el motivo por el cual fue necesario utilizar un subconjunto reducido de imágenes, fue la gestión de las cuotas de los *VLM*. Si bien *Google AI Studio* ofrecía acceso gratuito a sus modelos, estos presentaban una restricción de 15 invocaciones por minuto y 20 invocaciones por día por proyecto. Para maximizar el uso de estos recursos, se implementó el código de evaluación de manera tal que, al alcanzar el límite de peticiones por minuto, la ejecución se suspendía durante 60 segundos antes de reanudar las llamadas a la API. Sin embargo, esta limitación impidió analizar más de 20 imágenes por día, lo que determinó el tamaño máximo del conjunto de evaluación.

Luego de procesar la totalidad de las imágenes y obtener los resultados individuales de *Recall*, *SPO-Semantic* y *Graph Score*, se procedió a promediar dichos valores sobre el conjunto. El reporte final con los resultados consolidados de la evaluación se presenta en la Tabla III.

Métrica (Promedio)	PSG vs. GT	VLM vs. GT	Delta (Δ)
$nR@k$	0.3000	0.0000	0.3000
Semantic Similarity	0.6693	0.4096	0.2597
Graph Score	0.7874	0.4453	0.3421

TABLE III

RESULTADOS COMPARATIVOS DEL DESEMPEÑO DE LOS MODELOS *PSG* Y *LLM* RESPECTO AL *Ground Truth* (GT).

Como se observó en la tabla anterior, el modelo *PSG* obtuvo mejores resultados en las tres métricas, destacándose particularmente en la que se consideró más robusta: *Graph Score*, donde alcanzó un *score* del 78%. Asimismo, fue en esta métrica donde se registró la mayor diferencia entre ambos modelos, con una ventaja de 34 puntos porcentuales a favor de *PSG*. Esta diferencia sugiere que el entrenamiento específico en grafos de escena del *PSG* resultó significativamente más efectivo para describir la estructura espacial de una escena en comparación con la capacidad de razonamiento generalista del *VLM* utilizado.

Por otro lado, se observó que el *VLM* obtuvo un valor de $mR@k$ igual a 0, lo que indica que no logró recuperar

tripleas que coincidieran exactamente con el *ground truth*. En contraste, el *PSG* alcanzó un valor de 0.3 en esta métrica, cercano a los resultados reportados en el conjunto de prueba por los autores en [4] para el modelo *one-stage PSG*.

Por último, la métrica en la que ambos modelos presentaron un comportamiento más similar fue *SPO-Semantic*. No obstante, incluso en este caso, el *PSG* superó al *VLM* por aproximadamente 26 puntos porcentuales. Una posible explicación es que las descripciones generadas por el *VLM* tienden a centrarse en atributos y a extenderse con información que aporta poco a la caracterización espacial de la imagen.

D. Caso de estudio: escena con alta densidad de objetos

Una de las situaciones que se consideraron un desafío para los modelos es que la imagen presente una gran densidad de objetos distintos que se relacionen entre sí, y el objetivo sea enfocarse en algún objeto puntual. Con el fin de evaluar este caso, se hizo inferencia con ambos modelos sobre la imagen de la figura 7.



Fig. 7. Cuarto desordenado con varias relaciones. Imagen utilizada para la evaluación comparativa cualitativa entre el modelo *PSG* y el *VLM*

A continuación se presenta la descripción hecha por el *VLM*.

Respuesta generada por el VLM

``This image depicts a cozy and functional contemporary bedroom or studio apartment, characterized by a neutral color palette of browns, beiges, and whites, accented with touches of red and blue. The room is divided into distinct zones: a sleeping area with a bed and nightstand, a study area featuring a wooden desk and two chairs, and a storage area with a tall bookshelf. Various personal items like books, magazines, a laptop, a guitar, and casual clothing are visible, giving the space a lived-in and personalized feel.'`

Por otro lado, la salida de la rama de *PSG* fue la siguiente.

Respuesta generada por el PSG

"The rug-merged is on the floor-other-merged. The chair is on the floor-other-merged. The bed is on the floor-other-merged. The chair is on the floor-other-merged. The light is beside the bed. The remote is on the floor-other-merged. The laptop is on the bed. The handbag is on the floor-other-merged. The chair is beside the table-merged. The clock is on the table-merged. The chair is beside the bed. The cabinet-merged is on the floor-other-merged. The chair is beside the table-merged. The light is on the floor-other-merged. The curtain is beside the shelf. The curtain is beside the bed. The remote is on the rug-merged. The bed is beside the light. The shelf is beside the curtain. The book is on the shelf. The clock is over the bed. The shelf is on the floor-other-merged. The light is on the cabinet-merged. The clock is over the floor-other-merged. The clock is beside the curtain. The blanket is on the bed. The chair is in front of the window-other. The chair is beside the shelf. The bed is beside the chair. The window-blind is over the table-merged. The remote is beside the remote. The handbag is beside the table-merged. The window-blind is in front of the window-other. The clock is over the light. The window-blind is on the window-other. The chair is in front of the window-other."

Tal como se observó en el ejemplo de la Sección VI-B3, la descripción generada por *PSG* incluyó una mayor cantidad de objetos y relaciones, proporcionando una caracterización más exhaustiva de la escena. Por otro lado, la descripción generada por el *VLM* resultó más verbosa y se enfocó en atributos de los elementos de la imagen. Si la tarea fuera, por ejemplo, determinar la ubicación de una repisa en la imagen, utilizando *PSG* sería posible inferir que se encuentra al lado de la cortina, mientras que el *VLM* ni siquiera menciona dicho objeto.

Este ejemplo pone en evidencia un caso de uso en el que un modelo basado en *PSG* es capaz de resolver una tarea específica y relevante en un contexto real, donde un *VLM* presentaría un desempeño inferior.

Otro aspecto importante a destacar es el relacionado con los costos. El uso de un *VLM* a través de *Google AI Studio* u otros servicios similares suele estar sujeto a esquemas de pago por uso o a cuotas gratuitas restrictivas, como las mencionadas previamente. En contraste, el modelo *PSG* constituye una solución *open-source*, lo que permite su ejecución en infraestructura local sin incurrir en costos asociados a la inferencia.

En esta misma línea, mientras que el *VLM* requiere una conexión a internet para realizar llamadas a la API, el modelo

PSG puede operar de forma completamente *offline*. Esta característica resulta especialmente relevante en aplicaciones en tiempo real o en entornos donde la latencia y la seguridad de los datos son factores críticos.

VII. LA NECESIDAD DE UN *open-set*: *Open PSG*

En esta sección se presenta el estado del arte en *Open-set Panoptic Scene Graph Generation (Open PSG)*. Este enfoque surge como una extensión de los modelos tradicionales de *PSG*, los cuales operan restringidos a un conjunto fijo de objetos y relaciones definidos durante el entrenamiento.

Esta limitación resulta significativa en escenarios reales, donde la variedad de objetos y relaciones posibles es prácticamente ilimitada. En este sentido, los modelos *closed-set* presentan una capacidad de generalización reducida, ya que no pueden reconocer ni describir entidades fuera de su vocabulario predefinido.

Por otro lado, los *Vision-Language Models (VLMs)* abordan naturalmente este problema gracias a su entrenamiento sobre grandes volúmenes de datos, lo que les permite generar descripciones abiertas y flexibles.

Sin embargo, como se discutió en secciones anteriores, estos modelos carecen de una representación estructurada explícita, lo que limita su capacidad para capturar relaciones espaciales precisas entre objetos.

En este contexto, *Open PSG* emerge como un intento de combinar la capacidad de generalización semántica de los *VLMs*, y la representación estructurada y explícita de relaciones propia de los modelos de grafos de escena. Para ello, estos modelos incorporan componentes multimodales que permiten asociar representaciones visuales con descripciones textuales abiertas, extendiendo así el espacio de categorías más allá del conjunto de entrenamiento.

En particular, en este trabajo se consideró el modelo propuesto en [5], el cual representa una de las aproximaciones más recientes en esta línea. No obstante, no fue posible realizar experimentos de inferencia con dicho modelo debido a problemas de compatibilidad entre versiones de librerías y dependencias del entorno de ejecución.

VIII. SOTA: OPENPSG

Para este análisis se consideró un modelo de *Open-set PSG* ya implementado [5].

En este caso, la arquitectura completa se compone de tres módulos principales, como se muestra en la Figure 8. Cada uno de estos bloques cumple un rol específico, permitiendo pasar de una representación visual de la imagen a un grafo de escena en un entorno *open-set*.

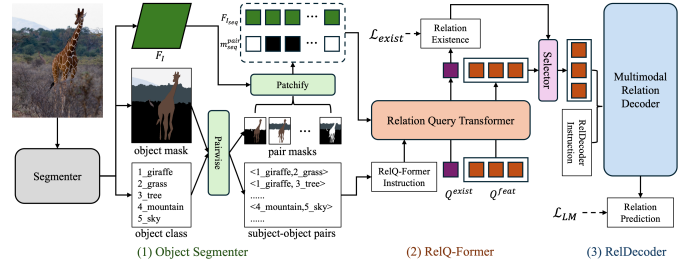


Fig. 8. Modelo Open PSG

Uno de los aspectos más relevantes de esta arquitectura es la incorporación de *LLMs* dentro del *pipeline*, lo cual constituye el principal mecanismo para habilitar el comportamiento *open-set*. En particular, estos modelos permiten enriquecer las predicciones visuales con conocimiento semántico externo, facilitando la identificación y descripción de objetos y relaciones que no fueron observados durante el entrenamiento.

En las siguientes subsecciones se detallan los distintos componentes del modelo y el flujo de información entre ellos.

A. Segmentador

El modelo cuenta con un *open-set panoptic segmenter* preentrenado, encargado de segmentar los objetos presentes en la imagen.

Este módulo recibe como entrada la imagen cruda y produce como salida las *features* visuales, denotadas como F_1 , tal como se detalló en la section III.

Es importante destacar que este enfoque representa una mejora respecto al modelo de *PSG* analizado en secciones anteriores.

Luego del módulo de segmentación, el flujo se divide en dos ramas que se ejecutan en paralelo.

- El módulo *Patchify* transforma el mapa de características F_1 en una secuencia de *tokens* para su procesamiento por el Transformer.

Para ello, aplica una convolución con *kernel* y *stride* p , que divide la representación en parches no superpuestos de tamaño $p \times p$. Como resultado, se obtiene una secuencia de vectores visuales.

En la figura 9 se ilustra este proceso.

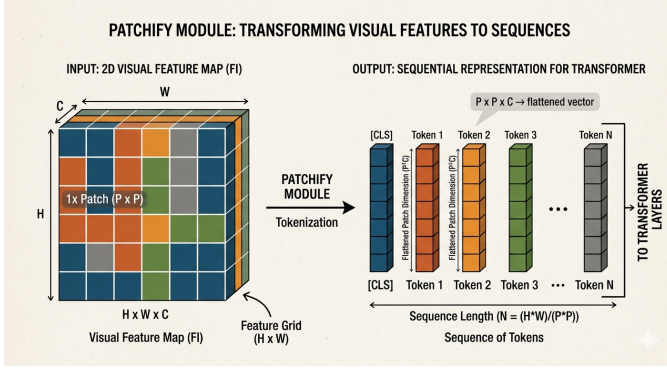


Fig. 9. Transformación hecha por el módulo Patchify sobre el mapa de características.

- El *Pairwise Module* tiene como objetivo construir parejas objeto-sujeto. Para esto, dado N objetos de la imagen, empareja cada objeto con los otros, así generando $N \cdot (N - 1)$ parejas de objetos diferentes.

B. Relation Query Transformer

El bloque *Relation Query Transformer (RelQ-Former)* extrae características de pares de objetos y filtra aquellos con relaciones relevantes antes de pasarlos a un modelo de lenguaje.

El módulo recibe tres elementos:

- un secuencia de FI_{seq} (*Visual Tokens*), donde cada token representa una region de la imagen
- c^{pair} categorías del par de objetos
- m_{seq}^{pair} y las máscaras asociadas a las regiones que pertenecen al par (*sujeto, objeto*)

El *RelQ-Former* utiliza *queries* aprendidas para realizar dos tareas en paralelo.

La primera es la extracción de características del par (*sujeto, objeto*). Para ello, emplea una instrucción textual que, tras ser tokenizada, guía un mecanismo de *masked cross-attention*, donde la atención sobre los *visual tokens* está restringida por la máscara m_{seq}^{pair} .

Las características obtenidas se refinan mediante una *Feed Forward Network*, produciendo el vector final F_{pair}^I .

La segunda tarea consiste en determinar si existe una relación entre los objetos. Para ello, se utiliza otra *query* guiada por una instrucción textual, cuya salida es procesada por una MLP de dos capas que predice la existencia de una relación.

C. Multimodal Relation Decoder

El *Multimodal Relation Decoder* es el módulo encargado de predecir la relación entre dos objetos. Como entrada recibe la representación visual del par de objetos obtenida en la etapa anterior, denotada como $F_I^{pair(i,j)}$.

Este módulo constituye el punto donde se incorpora un *Large Language Model (LLM)* dentro del pipeline de OpenPSG. A partir de las características visuales y de instrucciones textuales, el decoder infiere las posibles relaciones entre el sujeto o_i y el objeto o_j .

Los autores proponen dos estrategias diferentes para realizar esta predicción de relaciones entre pares (*sujeto, objeto*).

1) *Generation Instruction*: El modelo utiliza una instrucción en lenguaje natural que guía al decoder a generar las relaciones para el par (*sujeto, objeto*).

Esta instrucción se tokeniza en embeddings F_{inst}^{gen} , que se concatenan con la representación visual:

$$r_{i,j} = Decoder(Concat(F_I^{pair(i,j)}, F_{inst}^{gen}))$$

El decoder genera las relaciones de forma autoregresiva como una secuencia de posibles interacciones.

Este enfoque tiende a favorecer relaciones frecuentes del conjunto de entrenamiento. Su uso en el *Multimodal Relation Decoder* se denomina **OpenPSG-G** (ver Fig. 10).

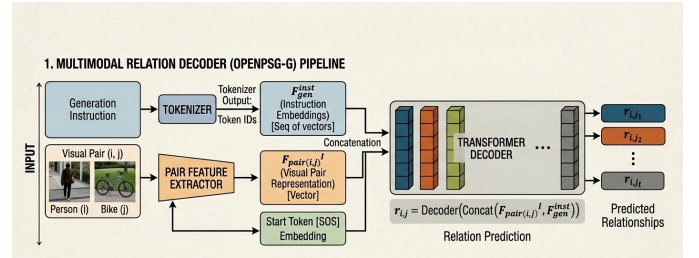


Fig. 10. Arquitectura del Multimodal Relation Decoder.

2) *Judgement Instruction*: A diferencia del enfoque anterior, en *Judgement Instruction* el modelo no genera relaciones, sino que evalúa relaciones candidatas. Dada una relación r_k , el decoder determina si el triplete (o_i, r_k, o_j) es válido mediante una respuesta binaria (Sí/No).

Esta variante se denomina **OpenPSG-J** y es el enfoque principal del trabajo. En consecuencia, cuando los autores se refieren a **OpenPSG**, aluden a **OpenPSG-J**. En la figura 11 se muestra un diagrama de esta estrategia.

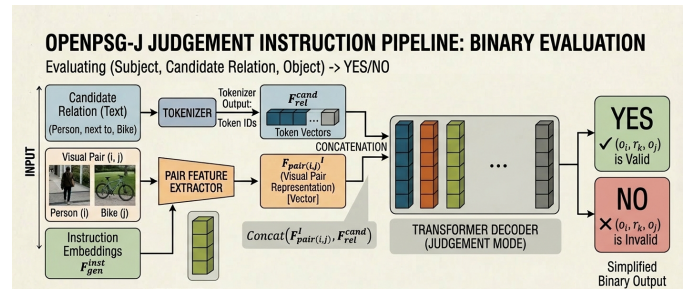


Fig. 11. Arquitectura de la estrategia con Judgment Instruction.

IX. CONCLUSIONES

En el presente informe analizamos la evolución desde el *Scene Graph Generation* tradicional hacia *Panoptic Scene Graph Generation (PSG)*, con el objetivo de comprender en qué medida una representación explícita y estructurada de la escena aporta ventajas frente a descripciones generadas por

modelos vision-language. La motivación inicial del informe planteaba una cuestión central: mientras que la detección de objetos responde a *qué* hay en una imagen, la comprensión de escena busca capturar *cómo* interactúan los elementos presentes. En ese sentido, los grafos de escena proponen una forma de representar no solo objetos, sino también sus relaciones y contexto.

Desde el punto de vista teórico, pudimos entender que la segmentación panóptica supone una mejora sustancial respecto del uso de *bounding boxes* en *SGG* tradicional. Al segmentar cada instancia con mayor precisión, *PSG* permite hacer una descripción más fiel de la escena, evitando el ruido. Además, las arquitecturas *one-stage* exploradas, como *PSGTR* y *PSG-Former*, muestran que es posible modelar de forma conjunta objetos y relaciones mediante mecanismos de atención y *queries* aprendidas, evitando parte de las limitaciones de los pipelines secuenciales. Este cambio resulta importante en escenas complejas, donde la localización precisa y la interacción entre múltiples elementos es el objetivo principal.

En la etapa experimental pudimos comprobar que la implementación de *PSG* provista por los autores de [4] fue efectivamente utilizable a partir del *checkpoint* provisto y los resultados que obtuvimos fueron consistentes con los reportados en el artículo original.

En cuanto a las descripciones generadas por los modelos, mientras que las del *VLM* tendieron a ser más verbosas y ricas en atributos, *PSG* produjo representaciones más estructuradas y orientadas a la relación espacial. Los resultados cuantitativos confirmaron la hipótesis de trabajo: *PSG* obtuvo mejor desempeño que el *VLM* en las tres métricas consideradas. En particular, destacó en *Graph Score*, donde la diferencia respecto del *VLM* fue la mayor, lo que sugiere que el entrenamiento específico en grafos de escena permite describir con mayor precisión la estructura relacional de una imagen. Asimismo, el hecho de que *PSG* alcanzara un valor de *recall* medio distinto de cero, mientras que el *VLM* no recuperó tripletas exactas respecto del *ground truth*, refuerza la idea de que los modelos especializados siguen siendo más adecuados para tareas que requieren exactitud espacial.

El caso de estudio con alta densidad de objetos fue especialmente ilustrativo: *PSG* permitió identificar relaciones entre elementos específicos de la escena, incluso cuando la imagen tenía alta densidad de objetos. En contraste, el *VLM* generó una descripción más general, útil para una lectura global de la escena, pero menos precisa cuando se requería localizar o vincular objetos particulares. Esto pone en evidencia que, aunque los *VLMs* son muy versátiles, no reemplazan completamente a un modelo especializado cuando la tarea exige un entendimiento explícito y estructurado de la escena.

Finalmente, la exploración de *OpenPSG* mostró que el problema sigue evolucionando hacia escenarios más amplios, donde ya no basta con un conjunto cerrado de categorías, sino que es necesario incorporar relaciones y conceptos abiertos. En conclusión, este trabajo permite afirmar que *PSG* constituye una solución más adecuada que un *VLM* cuando el objetivo principal es modelar con precisión la estructura interna de

una imagen. Lejos de competir en todos los contextos, ambos enfoques resultan complementarios: los *VLMs* aportan flexibilidad lingüística y capacidad descriptiva general, mientras que *PSG* ofrece una representación explícita y precisa de la escena, especialmente valiosa en aplicaciones donde la interpretación espacial detallada es el foco.

X. TRABAJO A FUTURO

El presente trabajo abre múltiples líneas de investigación que permiten profundizar tanto en los aspectos metodológicos como en el análisis empírico de los modelos estudiados.

En primer lugar, resulta fundamental completar la implementación y evaluación del modelo de *Open-set PSG* propuesto en [5]. La imposibilidad de ejecutar inferencia debido a incompatibilidades de dependencias impidió validar empíricamente la integración entre generalización semántica abierta y representación estructurada. Un análisis exhaustivo de este modelo permitiría cuantificar en qué medida los enfoques *open-set* reducen la brecha entre la flexibilidad de los *VLMs* y la precisión relacional de los modelos de *PSG*. En particular, sería relevante evaluar su desempeño bajo las métricas propuestas (*Graph Score* y *SPO-Semantic*) y compararlo directamente con los modelos *closed-set*.

En segundo lugar, las restricciones impuestas por el acceso a *VLMs* limitaron el tamaño de la muestra, lo cual introduce varianza en las estimaciones, un estudio a mayor escala permitiría realizar análisis estadísticos más rigurosos.

Un punto que sería interesante explorar es hacer un testeo más exhaustivo de las capacidades descriptivas de los *VLMs*, ajustando el prompt para tareas más orientadas a la ubicación espacial de objetos o interacciones complejas, buscando que se acerque más a las capacidades del *PSG*.

Otra dirección relevante es la exploración de arquitecturas híbridas que integren explícitamente *PSG* y *VLMs*. En lugar de considerarlos como enfoques alternativos, se podrían diseñar sistemas en los que los *VLMs* actúen como módulos de refinamiento semántico sobre grafos generados, o donde los grafos funcionen como representaciones intermedias estructuradas para guiar la generación de lenguaje.

REFERENCES

- [1] COCO Consortium. Common objects in context (coco) dataset. <https://cocodataset.org/>, 2014. Accessed: 2026-03-22.
- [2] Gemini Team and Google. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025.
- [3] Alexander Kirillov, Kaiming He, Ross Girshick, Carsten Rother, and Piotr Dollár. Panoptic segmentation. *arXiv preprint arXiv:1801.00868*, 2018.
- [4] Jingkan Yang, Yi Zhe Ang, Zujin Guo, Kaiyang Zhou, Wayne Zhang, and Ziwei Liu. Panoptic scene graph generation. *arXiv preprint arXiv:2207.11247*, 2022.
- [5] Zijian Zhou, Zheng Zhu, Holger Caesar, and Miaojing Shi. Openpsg: Open-set panoptic scene graph generation via large multimodal models. *arXiv preprint arXiv:2407.11213*, 2024.